Manual do Programador

Arquitectura, código, API, base de dados e implantação

Guia técnico para quem vai instalar, manter ou evoluir o LeeVida: como o sistema está organizado, como correr o projecto localmente, como funcionam a autenticação, as regras de negócio, as notificações, a auditoria e as funcionalidades de inteligência artificial, e como o colocar em produção.

Projecto: Trabalho de Conclusão de Curso — Engenharia Informática, ISPTEC
Autora: Nancylee Danila Tome Jamba · **Orientador:** Judson Paiva
Versão do documento: 1.0 · Novembro de 2026
Repositório de artefactos: github.com/RuiYuriAfricano/leevida-tcc

Índice

1. Introdução
2. Visão geral do sistema
3. Arquitectura
4. Tecnologias e versões
5. Estrutura do código
6. Preparar o ambiente de desenvolvimento
   Pré-requisitos · Base de dados · Variáveis de ambiente · Arranque · Contas de demonstração
7. Backend (API NestJS)
   Módulos · Ciclo de vida de um pedido · Autenticação e autorização · Regras de negócio · Notificações em tempo real · Auditoria
8. Referência da API
9. Base de dados
   Modelo de domínio · Enumerações · Migrações · Dados de demonstração
10. Módulo de inteligência artificial
    Componentes · Regras clínicas · Fiabilidade · Privacidade · Configuração
11. Frontend (React)
    Estrutura · Rotas e perfis · Estado global · Cliente da API · Exportações
12. Testes
13. Segurança
14. Implantação em produção
15. Guias práticos de evolução
16. Resolução de problemas

1. Introdução

O LeeVida é um sistema web de prontuário clínico concebido para pequenas clínicas angolanas, com a Clínica EmiPharma (Caála, Huambo) como caso de estudo. Este manual destina-se a programadores e técnicos de informática que precisem de instalar o sistema, corrigir erros, acrescentar funcionalidades ou colocá-lo em produção. Pressupõe familiaridade com TypeScript, Node.js, React e bases de dados relacionais.

O código está organizado como um monorepo com dois projectos independentes:

Pasta	Conteúdo
`leevida-backend/`	API REST em NestJS, com Prisma (ORM) e PostgreSQL.
`leevida-front-end/`	Aplicação web (Single Page Application) em React, Vite e Tailwind CSS.

> **Convenções deste manual**
> Os caminhos de ficheiros são relativos à raiz do monorepo. Os comandos de terminal são escritos para bash (Git Bash no Windows). As rotas da API omitem o prefixo `/api/v1`, salvo indicação em contrário.

2. Visão geral do sistema

O sistema serve quatro perfis de utilizador, cada um com um conjunto fechado de permissões aplicado no servidor:

Perfil	O que faz no sistema
Administrador	Gere as contas dos profissionais, repõe senhas, desactiva contas, consulta o registo de auditoria e vê os indicadores globais.
Enfermeiro	Regista pacientes (o registo cria também a conta do paciente), faz a triagem com sinais vitais, sintomas e prioridade de Manchester, e acompanha o estado dos exames sem ver os resultados.
Médico	Gere a agenda, aceita ou rejeita consultas, atende a fila por prioridade, regista diagnósticos e códigos CID-10, emite e assina prescrições, solicita exames e regista resultados, emite justificativos e usa o assistente de IA.
Paciente	Não se auto-regista: entra com as credenciais dadas pelo enfermeiro. No portal marca consultas e consulta o seu histórico, receitas e exames.

Às funcionalidades clínicas juntam-se notificações em tempo real, exportação para Excel e PDF, registo de auditoria de todas as acções críticas e seis funcionalidades de inteligência artificial de apoio à decisão (secção 10).

3. Arquitectura

O LeeVida segue uma arquitectura cliente-servidor em três camadas. O navegador comunica apenas com a API, nunca directamente com a base de dados nem com o serviço de IA.

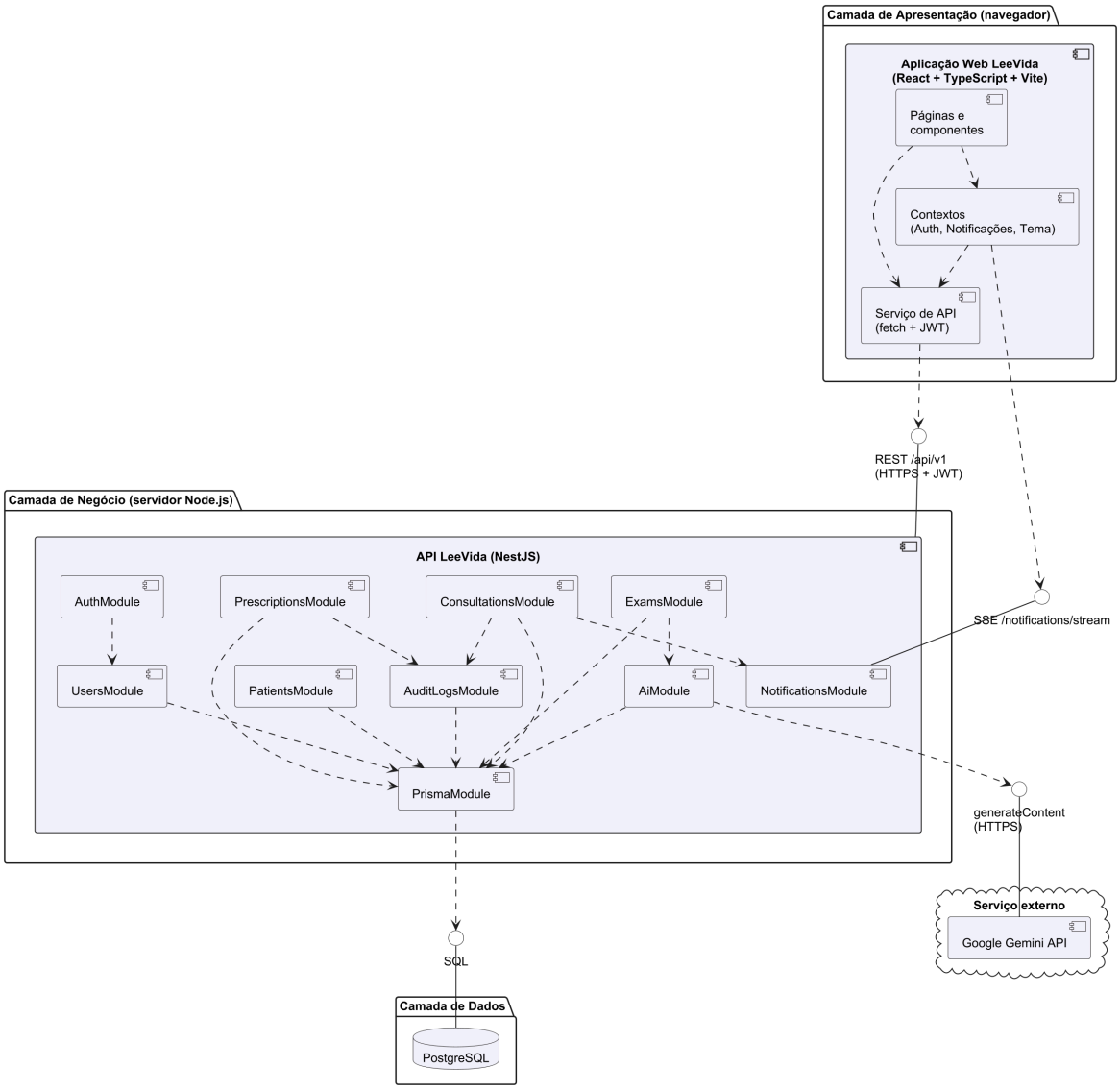


Diagrama de componentes do LeeVida (UML 2.5.1).

Camada	Responsabilidade
Apresentação	SPA em React: páginas por perfil, componentes reutilizáveis e contextos de estado (autenticação, notificações, tema e mensagens). Fala com a API por um cliente HTTP único, que injecta o token JWT e renova a sessão automaticamente.
Negócio	API NestJS organizada em módulos por domínio. Concentra validação, autenticação, autorização por perfil, regras de negócio, auditoria, notificações (Server-Sent Events) e a integração com a IA.
Dados	PostgreSQL, acedido exclusivamente pelo `PrismaModule`. O esquema e as migrações estão em `leevida-backend/prisma/`.

Camada	Responsabilidade
Serviço externo	Google Gemini API, chamada apenas pelo `AiModule`, com dados anonimizados. É opcional: sem chave, o sistema funciona com regras locais.

4. Tecnologias e versões

Componente	Versão	Uso
Node.js	20 ou superior	Execução da API e ferramentas de build
TypeScript	5.x	Linguagem do backend e do frontend
NestJS	11	Framework da API (módulos, injecção de dependências, guards, pipes)
Prisma ORM	6.19	Esquema, cliente tipado e migrações
PostgreSQL	16 ou superior	Base de dados relacional
Passport + JWT, bcrypt	—	Autenticação e hash de senhas
Helmet, @nestjs/throttler	—	Cabeçalhos de segurança e limitação de pedidos
React	18.3	Interface
React Router	7	Navegação e rotas protegidas por perfil
Vite	5.4	Servidor de desenvolvimento e build
Tailwind CSS	3.4	Estilos
lucide-react, ExcelJS, jsPDF	—	Ícones e exportação para Excel e PDF
Jest / Vitest	30 / 4	Testes do backend / frontend
Google Gemini API	—	Funcionalidades de IA (opcional)

5. Estrutura do código

Backend — `leevida-backend/`

```
prisma/
  schema.prisma        modelo de dados (fonte de verdade)
  migrations/          migrações SQL versionadas
  seed.ts              dados de demonstração
src/
  main.ts              arranque: Helmet, CORS, versionamento, ValidationPipe, Swagger
  app.module.ts        registo dos módulos e do ThrottlerGuard global
  config/              leitura e validação das variáveis de ambiente
  prisma/              PrismaService / PrismaModule (global)
  common/              decorators (@Public, @Roles, @CurrentUser), RolesGuard,
                       filtro de excepções, interceptor de logs, política de senhas
  auth/                login, refresh, logout, troca de senha, estratégia JWT
  users/               gestão de utilizadores (admin) e perfil próprio
  patients/            registo de pacientes e emissão de credenciais
  consultations/       consultas, triagem, marcação, fila de espera
  prescriptions/       emissão e assinatura de receitas
  exams/               pedido de exames e registo de resultados
  doctor-schedules/    agenda de disponibilidade dos médicos
  justifications/      justificativos médicos
  notifications/       notificações (persistidas + SSE)
  audit-logs/          registo de auditoria
  ai/                  integração Gemini, regras clínicas, contexto anonimizado
test/                  testes ponta a ponta (Jest)
```

Cada módulo de domínio segue o padrão NestJS: `*.module.ts` (ligações), `*.controller.ts` (rotas e perfis), `*.service.ts` (regras de negócio e acesso ao Prisma) e `dto/` (objectos de entrada validados com `class-validator`).

Frontend — `leevida-front-end/src/`

```
App.tsx              rotas e protecção por perfil (ProtectedRoute)
main.tsx             arranque
layouts/             DashboardLayout, Sidebar, Topbar
pages/               uma página por área (Dashboard, Patients, Consultations,
                     Prescriptions, Exams, Nurse, Agenda, History, Admin,
                     Logs, PatientPortal, Profile, Login)
components/          AiAssistant, AiPatientSummary, PrescriptionSafetyModal,
                     ExamInsightModal, TriageBadge, ExportMenu, ConfirmModal, …
contexts/            AuthContext, NotificationContext, ThemeContext, ToastContext
services/apiClient.ts   cliente HTTP (token, renovação de sessão, erros)
services/api/        um ficheiro por recurso da API (patients.ts, exams.ts, ai.ts, …)
hooks/               useDoctorDirectory
utils/               exportação Excel/PDF, impressão de documentos, formatação
types/               tipos partilhados
```

6. Preparar o ambiente de desenvolvimento

6.1 Pré-requisitos

- Node.js 20+ e npm.
- PostgreSQL 16+ instalado localmente, **ou** Docker Desktop (o projecto inclui um `docker-compose.yml`).
- Git.
- Opcional: uma chave da API Gemini (`aistudio.google.com/apikey`) para activar a IA.

6.2 Base de dados

Com Docker, a partir de `leevida-backend/`:

```
docker compose up -d     # PostgreSQL 16 na porta 5433, base leevida_db
```

Com PostgreSQL nativo, basta criar uma base e apontar `DATABASE_URL` para ela.

6.3 Variáveis de ambiente

Copiar `leevida-backend/.env.example` para `.env` e preencher:

Variável	Significado
`NODE_ENV`, `PORT`	Ambiente e porta da API (por omissão 3000).
`DATABASE_URL`	Ligação PostgreSQL, por exemplo `postgresql://utilizador:senha@localhost:5433/leevida_db?schema=public`.
`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`	Segredos de assinatura dos tokens. Usar valores longos e aleatórios, diferentes entre si.
`JWT_ACCESS_EXPIRES_IN`, `JWT_REFRESH_EXPIRES_IN`	Validade dos tokens (por omissão `15m` e `7d`).
`CORS_ORIGIN`	Origem autorizada do frontend, por exemplo `http://localhost:5173`.
`TRUST_PROXY`	Número de proxies de confiança em produção (vazio em desenvolvimento). Necessário para o IP correcto na auditoria.
`CLINIC_TIMEZONE`	Fuso horário da clínica (`Africa/Luanda`), usado para validar marcações contra a agenda.
`GEMINI_API_KEY`	Chave da API Gemini. Sem chave, a IA fica desactivada e o sistema usa só as regras locais.
`GEMINI_MODEL`, `GEMINI_FALLBACK_MODELS`	Modelo principal e lista de modelos de reserva, separados por vírgulas.

No frontend, `leevida-front-end/.env` define `VITE_API_URL` (por omissão `http://localhost:3000/api/v1`).

> **Nunca versionar o ficheiro .env**
> Contém a ligação à base de dados, os segredos JWT e a chave da IA. O repositório só deve conter o `.env.example`.

6.4 Arranque

```
# API
cd leevida-backend
npm install
npm run prisma:migrate     # aplica as migrações
npm run db:seed            # dados de demonstração
npm run start:dev          # http://localhost:3000/api/v1  ·  Swagger em /api/docs

# Aplicação web (noutro terminal)
cd leevida-front-end
npm install
npm run dev                # http://localhost:5173
```

6.5 Scripts úteis

Comando	Função
`npm run start:dev`	API com recarregamento automático
`npm run build` / `start:prod`	Compilar a API para `dist/` e correr a versão compilada
`npm run prisma:migrate`	Criar ou aplicar migrações em desenvolvimento
`npm run prisma:studio`	Interface gráfica para inspeccionar a base de dados
`npm run db:seed`	Repor os dados de demonstração
`npm test` (backend)	Testes unitários com Jest
`npm run lint`	ESLint (backend e frontend)
`npm run typecheck` (frontend)	Verificação de tipos sem gerar ficheiros
`npm test` (frontend)	Testes com Vitest

6.6 Contas de demonstração

Criadas pelo `db:seed`, todas com a senha `123456`. Servem apenas para desenvolvimento e devem ser removidas em produção.

E-mail	Perfil
`admin@leevida.ao`	Administrador
`medico@leevida.ao`	Médico (Clínica Geral)
`beatriz@leevida.ao`	Médico (Pediatria)
`enfermeira@leevida.ao`, `carlos@leevida.ao`	Enfermeiro

E-mail	Perfil
`paciente@leevida.ao`	Paciente

7. Backend (API NestJS)

7.1 Módulos

Módulo	Responsabilidade
`AuthModule`	Login, renovação e fim de sessão, troca de senha. Regista os guards globais `JwtAuthGuard` e `RolesGuard`.
`UsersModule`	Contas dos profissionais (só administrador) e perfil do próprio utilizador.
`PatientsModule`	Registo, edição e arquivo de pacientes; cria a conta de acesso do paciente com senha provisória.
`ConsultationsModule`	Consultas normais e de retorno, triagem, marcação pelo paciente, aceitar/rejeitar/iniciar/concluir, fila de espera ordenada por prioridade.
`PrescriptionsModule`	Emissão de receitas e assinatura pelo médico emissor.
`ExamsModule`	Pedidos de exames, registo de resultados, análise automática (regras e IA), ocultação de resultados ao enfermeiro.
`DoctorSchedulesModule`	Agenda semanal de cada médico, médicos disponíveis e lista pública de especialidades.
`JustificationsModule`	Justificativos médicos associados a uma consulta.
`NotificationsModule`	Notificações por utilizador ou por perfil, persistidas e enviadas em tempo real por SSE.
`AuditLogsModule`	Registo e consulta da auditoria; regista também as exportações feitas no navegador.
`AiModule`	Funcionalidades de IA (secção 10).
`PrismaModule`	Serviço Prisma global, partilhado por todos os módulos.

7.2 Ciclo de vida de um pedido

1. **Middleware** (`main.ts`): Helmet (cabeçalhos de segurança), `cookie-parser`, contexto do pedido, compressão (excepto SSE) e CORS restrito a `CORS_ORIGIN`.
2. **Guards globais**, por esta ordem: `ThrottlerGuard` (100 pedidos/min por omissão), `JwtAuthGuard` (exige token, salvo rotas com `@Public()`) e `RolesGuard` (compara o perfil do utilizador com `@Roles(...)`).
3. **ValidationPipe global**: valida o DTO, rejeita campos desconhecidos (`whitelist` + `forbidNonWhitelisted`) e converte tipos.
4. **Controller → Service**: o serviço aplica as regras de negócio, usa o Prisma, regista a auditoria e emite notificações.
5. **Resposta**: o `LoggingInterceptor` regista o pedido e o `HttpExceptionFilter` uniformiza os erros.

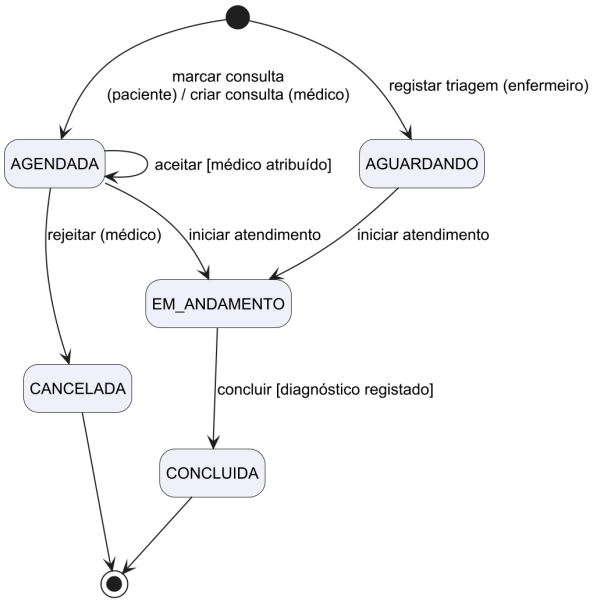
Todas as rotas ficam sob `/api/v1` (prefixo global e versionamento por URI). A documentação interactiva Swagger está em `/api/docs`.

7.3 Autenticação e autorização

- **Login** (`POST /auth/login`): valida e-mail e senha (bcrypt) e devolve um **access token** (15 min) no corpo. Guarda um **refresh token** (7 dias) num cookie `httpOnly` , inacessível a JavaScript.
- **Renovação** (`POST /auth/refresh`): o refresh token é rodado a cada uso, e a base de dados guarda apenas o seu hash (`User.hashedRefreshToken`). O frontend chama-o automaticamente quando recebe 401 e quando a página é recarregada.
- **Senhas provisórias**: contas criadas pelo administrador ou pelo enfermeiro, e senhas repostas, recebem uma senha gerada sem caracteres ambíguos e `mustChangePassword = true` . O utilizador tem de a trocar no primeiro login.
- **Política de senhas** (`common/password-policy.ts`): mínimo de 8 caracteres, com pelo menos uma letra e um número.
- **Decorators**: `@Public()` dispensa autenticação; `@Roles(Role.MEDICO, …)` restringe por perfil (ao nível da classe ou do método); `@CurrentUser()` injecta o utilizador autenticado.
- **Contas desactivadas** (`deactivatedAt`) não conseguem autenticar-se. As contas nunca são apagadas, para preservar a autoria clínica e a auditoria.

7.4 Regras de negócio principais

- Só enfermeiro e administrador registam pacientes. O registo cria a conta do paciente; o paciente nunca se auto-regista.
- Só médico e administrador criam consultas normais e de retorno. A triagem (enfermeiro) cria a consulta em estado `AGUARDANDO` .
- Transições de estado da consulta: `AGENDADA` → aceitar (atribui o médico) ou rejeitar (`CANCELADA`); `AGENDADA` / `AGUARDANDO` → iniciar (`EM_ANDAMENTO`) → concluir (`CONCLUIDA`). A API recusa transições inválidas.
- A fila de espera do médico ordena por prioridade de triagem (vermelho → azul) e depois por hora de chegada.
- O paciente só pode marcar com médicos que tenham agenda activa na especialidade escolhida, dentro do horário definido (validado no fuso `CLINIC_TIMEZONE`).
- Só o médico que emitiu uma receita a pode assinar.
- O enfermeiro nunca vê resultados de exames: a listagem remove `result` , `resultDate` e a análise da IA quando o pedido vem de um `ENFERMEIRO` . Só médico e administrador registam resultados.
- Um justificativo médico só pode ser emitido a partir de uma consulta `EM_ANDAMENTO` ou `CONCLUIDA` .
- Os pacientes não se apagam: são arquivados (`archivedAt`) e saem das listagens.



Máquina de estados da consulta.

7.5 Notificações em tempo real

O `NotificationsService` grava cada notificação na tabela `notifications` e envia-a pelo canal **Server-Sent Events** `GET /notifications/stream`. As notificações podem ser dirigidas a um utilizador (`notifyUser`) ou a um perfil inteiro (`notifyRole`, por exemplo "novo paciente em triagem" para todos os médicos). O `NotificationContext` do frontend mantém a ligação aberta e actualiza o sino da barra superior.

> **Limitação conhecida**
> As ligações SSE vivem na memória do processo. Com mais do que uma instância da API (escalonamento horizontal), é preciso um canal partilhado, como Redis pub/sub.

7.6 Auditoria

O `AuditLogsService.log(actor, acção, recurso, id)` grava o utilizador, o nome, a acção, o recurso, o identificador, a data/hora e o IP. É chamado pelos serviços em todas as acções críticas: autenticação, consultas, receitas, exames, consultas ao prontuário, gestão de utilizadores, exportações e cada utilização da IA. Exemplos de acções: `LOGIN`, `CONSULTOU_PRONTUARIO`, `ACEITOU_CONSULTA`, `FEZ_UPLOAD_RECEITA`, `ASSINOU_RECEITA_COM_ALERTA_IA`, `IA_SUGERIU_PRIORIDADE`. A tabela usa `onDelete: Restrict` sobre o utilizador, por isso os registos não se perdem.

8. Referência da API

Prefixo `/api/v1`. Todas as rotas exigem `Authorization: Bearer <token>`, excepto as marcadas como públicas. Os corpos e as respostas detalhados estão no Swagger (`/api/docs`).

Método	Rota	Perfis	Função
POST	`/auth/login`	público	Iniciar sessão
POST	`/auth/refresh`	público (cookie)	Renovar o access token
POST	`/auth/logout`	autenticado	Terminar sessão
POST	`/auth/change-password`	autenticado	Trocar a senha
GET/PATCH	`/users/me`	autenticado	Perfil próprio (nome, foto)
GET/POST	`/users`	Admin	Listar / criar utilizadores
GET/PATCH/DELETE	`/users/:id`	Admin	Ver / editar / desactivar
POST	`/users/:id/reset-password`	Admin	Repor senha (provisória)
GET/PATCH	`/patients/me`	Paciente	Dados próprios
GET	`/patients` , `/patients/:id`	Admin, Médico, Enfermeiro	Listar / ver paciente
POST/PATCH	`/patients` , `/patients/:id`	Admin, Enfermeiro	Registar (cria a conta) / editar
DELETE	`/patients/:id`	Admin, Enfermeiro	Arquivar paciente
GET	`/consultations` , `/:id`	Admin, Médico, Enfermeiro	Listar / ver consultas
GET	`/consultations/me`	Paciente	Consultas próprias
GET	`/consultations/waiting-queue`	Médico, Admin	Fila ordenada por prioridade
POST	`/consultations`	Médico, Admin	Criar consulta (normal ou retorno)
POST	`/consultations/triage`	Enfermeiro, Admin	Registar triagem (cria consulta AGUARDANDO)
POST	`/consultations/book`	Paciente	Marcar consulta
PATCH	`/consultations/:id/accept` , `/reject`	Médico	Aceitar / rejeitar

Método	Rota	Perfis	Função
PATCH	`/consultations/:id/start`, `/complete`	Médico, Admin	Iniciar / concluir (diagnóstico, CID-10)
GET/POST	`/prescriptions`, `/:id`	Médico, Admin	Listar / emitir receitas
PATCH	`/prescriptions/:id/sign`	Médico, Admin	Assinar (só o emissor)
GET	`/prescriptions/me`	Paciente	Receitas próprias
GET	`/exams`, `/exams/:id`	Admin, Médico, Enfermeiro	Listar exames (sem resultados para o enfermeiro)
POST	`/exams`	Médico, Admin	Solicitar exame
PATCH	`/exams/:id/result`	Médico, Admin	Registar resultado (dispara a análise)
GET	`/exams/me`	Paciente	Exames próprios
GET/PUT	`/doctor-schedules/me`	Médico	Ver / gravar a agenda
GET	`/doctor-schedules/available`	autenticado	Médicos com agenda activa
GET	`/doctor-schedules/specialties`	público	Especialidades disponíveis
POST	`/justifications`	Médico	Emitir justificativo
GET	`/justifications/me`	Paciente	Justificativos próprios
GET	`/notifications/me`	autenticado	Notificações
SSE	`/notifications/stream`	autenticado	Canal em tempo real
PATCH	`/notifications/read-all`, `/:id/read`	autenticado	Marcar como lidas
DELETE	`/notifications`	autenticado	Limpar notificações
GET	`/audit-logs`	Admin	Consultar auditoria
POST	`/audit-logs/exports`	todos	Registar uma exportação feita no navegador
GET	`/ai/status`	Médico, Enfermeiro, Admin	Indica se a IA está activa
POST	`/ai/triage-priority`	Enfermeiro, Admin	Sugestão de prioridade de triagem
GET	`/ai/patients/:id/summary`	Médico	Resumo pré-consulta
POST	`/ai/prescription-assistant`	Médico	Mensagem para o assistente

Método	Rota	Perfis	Função
GET/DELETE	`/ai/conversations[/:id]`	Médico	Histórico de conversas
POST	`/ai/prescriptions/:id/check`	Médico	Verificação da receita
POST	`/ai/exams/:id/interpret`	Médico	Interpretação de exame
POST	`/ai/icd10`	Médico	Sugestão de códigos CID-10

> **Limites**
> Os endpoints `/ai/*` têm um limite próprio de 20 pedidos por minuto por utilizador; o resto da API tem 100 pedidos por minuto.

9. Base de dados

9.1 Modelo de domínio

O ficheiro `prisma/schema.prisma` é a fonte de verdade. O diagrama seguinte representa-o em UML.

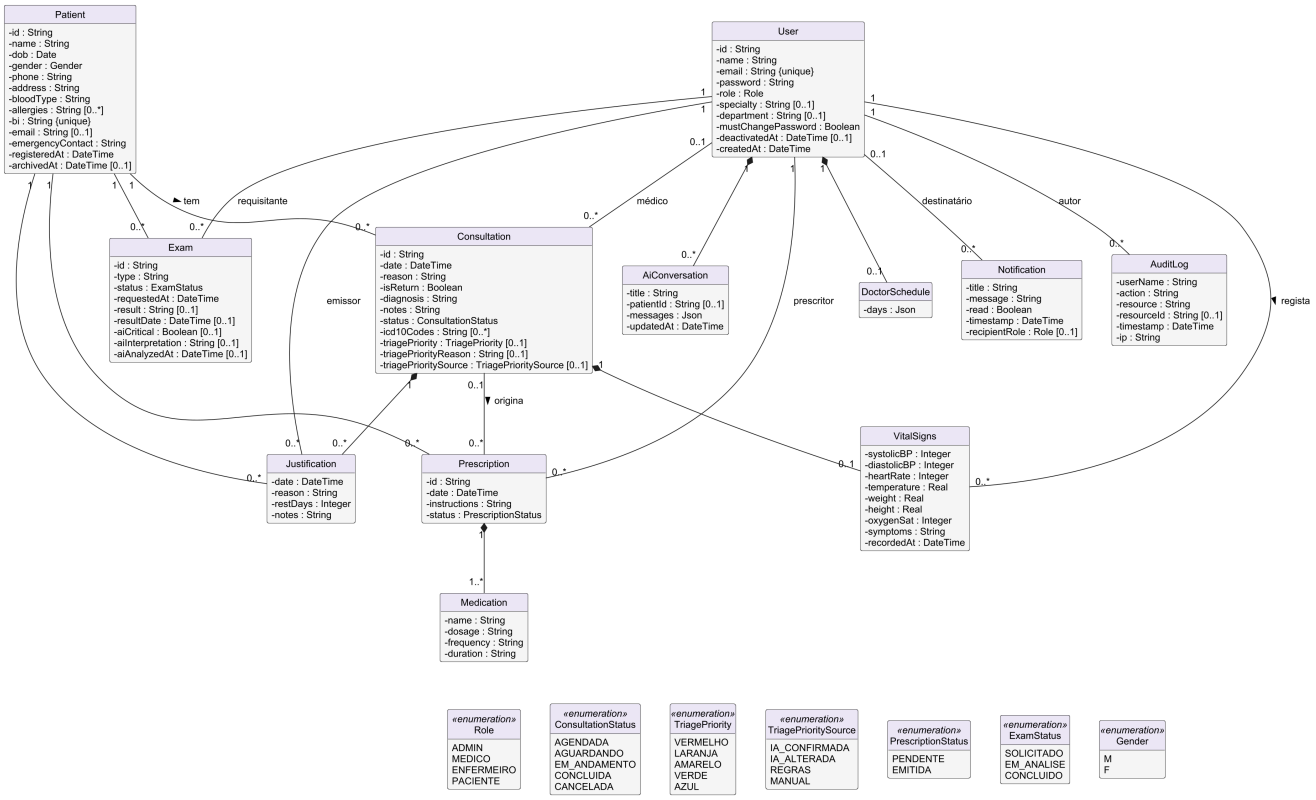


Diagrama de classes do domínio (equivalente ao esquema Prisma).

Modelo (tabela)	Descrição e regras de integridade
`User` (users)	Todos os utilizadores; `role` define o perfil; `email` único; soft delete por `deactivatedAt`.
`Patient` (patients)	Ficha do paciente; `bi` e `email` únicos; `allergies` é uma lista; arquivado por `archivedAt`.
`Consultation`	Consulta (normal, retorno ou triagem); estado, diagnóstico, notas, CID-10, prioridade de triagem e a sua origem. Paciente com `onDelete: Restrict`.
`VitalSigns`	Sinais vitais e sintomas de uma consulta (1:0..1, apagados com a consulta).
`Prescription` / `Medication`	Receita e os seus medicamentos (composição, cascade); estado `PENDENTE` ou `EMITIDA`.
`Exam`	Pedido, estado, resultado e campos da análise automática (`aiCritical` , `aiFlags` , `aiInterpretation` , `aiAnalyzedAt`).
`DoctorSchedule`	Disponibilidade semanal do médico, em JSON `{dia: {active, start, end}}`.
`Justification`	Justificativo médico ligado a uma consulta.
`Notification`	Para um utilizador (`recipientUserId`) ou para um perfil (`recipientRole`).

Modelo (tabela)	Descrição e regras de integridade
`AuditLog`	Registo de auditoria, imutável na prática.
`AiConversation`	Conversas do assistente, privadas de cada médico; mensagens em JSON.

9.2 Enumerações

Enumeração	Valores
`Role`	ADMIN, MEDICO, ENFERMEIRO, PACIENTE
`ConsultationStatus`	AGENDADA, AGUARDANDO, EM_ANDAMENTO, CONCLUIDA, CANCELADA
`TriagePriority`	VERMELHO, LARANJA, AMARELO, VERDE, AZUL
`TriagePrioritySource`	IA_CONFIRMADA, IA_ALTERADA, REGRAS, MANUAL
`PrescriptionStatus`	PENDENTE, EMITIDA
`ExamStatus`	SOLICITADO, EM_ANALISE, CONCLUIDO
`Gender`	M, F

9.3 Migrações

Migração	Conteúdo
`20260810234452_init`	Esquema inicial
`20260810234835_add_refresh_token`	Hash do refresh token no utilizador
`20261029154339_soft_delete_and_password_policy`	Desactivação de contas, arquivo de pacientes, senha provisória
`20261029163636_ai_clinical_support`	Prioridade de triagem, CID-10, análise de exames
`20261029170404_ai_conversations`	Conversas do assistente de IA

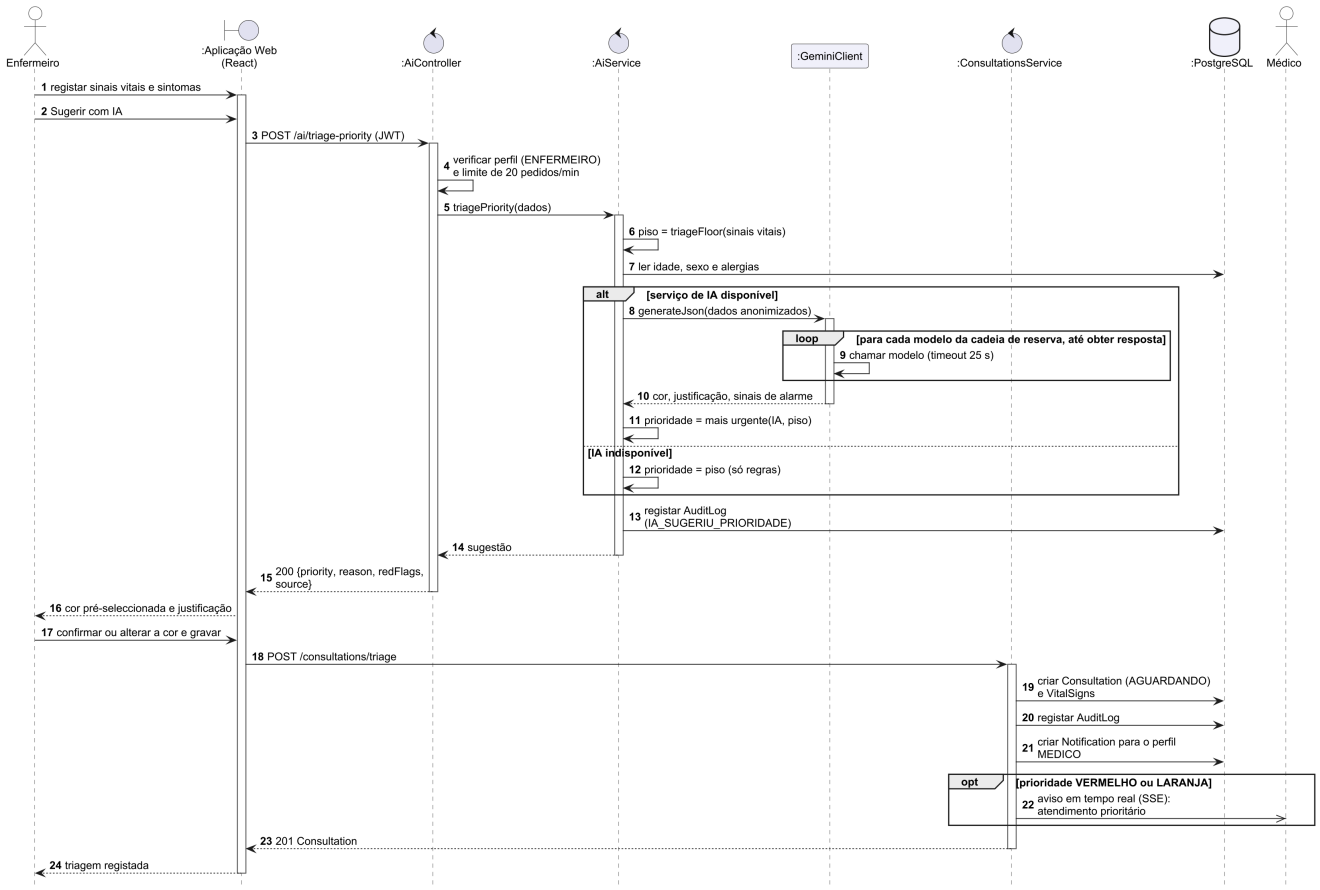
Em desenvolvimento usa-se `npm run prisma:migrate`, que cria uma nova migração a partir das alterações ao `schema.prisma`. Em produção usa-se `npx prisma migrate deploy`, que só aplica as migrações existentes. Nunca se editam migrações já aplicadas.

10. Módulo de inteligência artificial

O `AiModule` implementa seis funcionalidades de apoio à decisão clínica com modelos Google Gemini: triagem, resumo pré-consulta, assistente de prescrição, verificação da receita, interpretação de exames e sugestão de CID-10. Princípio central: **a IA sugere, o profissional decide**, e regras determinísticas locais funcionam mesmo sem IA.

10.1 Componentes

Ficheiro	Responsabilidade
`ai.controller.ts`	Endpoints `/ai/*`, perfis e limite de 20 pedidos/min.
`ai.service.ts`	As seis funcionalidades: combina regras e IA, grava os resultados e regista a auditoria.
`gemini.client.ts`	Comunicação com a API Gemini: cadeia de modelos de reserva, timeout de 25 s por modelo, pausa de 3 min de modelos que falham, respostas JSON com esquema.
`clinical-context.ts`	Constrói o contexto clínico **anonimizado**: idade, sexo, grupo sanguíneo, alergias, sinais vitais, diagnósticos, medicação e exames recentes.
`clinical-rules.ts`	Regras determinísticas: prioridade mínima por sinais vitais (`triageFloor`), conflitos de alergia por classe de fármaco (`allergyConflicts`) e valores laboratoriais de pânico.



Sequência de um pedido de IA: triagem com prioridade mínima por regras.

10.2 Regras clínicas (resumo)

- **Triagem**: a prioridade final é sempre a mais urgente entre a da IA e a calculada pelos sinais vitais (por exemplo, SpO_2 < 90% impõe no mínimo laranja). A IA pode subir a prioridade, nunca descê-la.
- **Alergias**: cruzamento por classes (penicilinas, AINE/aspirina, sulfonamidas, …), sem falsos positivos por semelhança de nome (por exemplo, "ácido fólico" não é confundido com "ácido acetilsalicílico").
- **Valores de pânico**: hemoglobina, plaquetas, leucócitos, potássio, sódio, glicemia e cálcio, com vírgula decimal, separadores de milhares e conversão de unidades. Prevalecem sobre a IA.

Os limiares estão todos em `clinical-rules.ts` para facilitar a revisão pela direcção clínica antes do uso em produção.

10.3 Fiabilidade

- Cadeia de modelos configurável (`GEMINI_MODEL` e `GEMINI_FALLBACK_MODELS`); um modelo que falha (sobrecarga, limite ou timeout) fica em pausa 3 minutos e passa para o fim da fila.
- Análise de exames em segundo plano, com novas tentativas após 30 s, 2 min e 5 min.
- Degradação graciosa: sem IA, a triagem usa as regras, a verificação cruza as alergias e o assistente mostra uma mensagem clara.
- Cache do resumo pré-consulta até 6 horas, invalidada quando o prontuário muda.
- Envio apenas das 20 últimas mensagens da conversa do assistente.

10.4 Privacidade

O modelo nunca recebe nome, BI, telefone, e-mail, morada, contacto de emergência, dados do médico ou credenciais. Um teste automático garante que o contexto enviado contém as alergias mas não estes dados. A chave existe apenas no servidor.

> **Antes de usar com dados reais**
> No plano gratuito da API Gemini, a Google pode usar o conteúdo dos pedidos para melhorar os seus produtos. Para dados de pacientes é obrigatório activar a facturação (plano pago) no projecto Google Cloud e garantir que o consentimento informado cobre o uso de IA.

11. Frontend (React)

11.1 Rotas e perfis

Rota	Página	Perfis
`/login`	LoginPage	público
`/dashboard`	DashboardPage	todos (conteúdo por perfil)
`/patients`, `/patients/:id`	PatientsPage, PatientDetailPage	Admin, Médico, Enfermeiro
`/consultations`	ConsultationsPage	Admin, Médico, Enfermeiro
`/prescriptions`	PrescriptionsPage	Admin, Médico
`/exams`	ExamsPage	Admin, Médico, Enfermeiro
`/history`	HistoryPage	Admin, Médico, Enfermeiro
`/nurse`	NursePage (triagem)	Enfermeiro, Admin
`/agenda`	AgendaPage	Médico
`/admin`, `/logs`	AdminPage, LogsPage	Admin
`/portal`	PatientPortalPage	Paciente
`/profile`	ProfilePage	todos

O componente `ProtectedRoute` (em `App.tsx`) redirecciona para `/login` sem sessão e bloqueia perfis não autorizados. Esta protecção é apenas de interface: a autorização efectiva é feita pela API.

11.2 Estado global

- **AuthContext**: utilizador autenticado, `login` / `logout`, troca de senha e `can(permissão)` para mostrar ou ocultar opções. Ao abrir a aplicação, recupera a sessão a partir do cookie de refresh.
- **NotificationContext**: lista de notificações e ligação SSE.
- **ThemeContext**: tema claro ou escuro.
- **ToastContext**: mensagens transitórias de sucesso e erro.

11.3 Cliente da API

`services/apiClient.ts` centraliza os pedidos: junta o access token (guardado só em memória), envia o cookie (`credentials: 'include'`), tenta uma renovação automática ao receber 401 e repete o pedido, e converte erros em `ApiError`. Cada recurso tem o seu módulo em `services/api/`, com funções tipadas.

11.4 Exportações e impressão

O `ExportMenu` gera ficheiros Excel (ExcelJS) e PDF (jsPDF + autoTable) no próprio navegador, a partir das colunas definidas em `utils/exportColumns.ts`, e regista cada exportação na auditoria.

`utils/printDocuments.ts` produz receitas e justificativos para impressão e assinatura.

11.5 Componentes de IA

`AiAssistant` (painel flutuante do médico), `AiPatientSummary` (resumo na ficha), `PrescriptionSafetyModal` (verificação antes de assinar), `ExamInsightModal` (interpretação) e `TriageBadge` (cor de Manchester). Todos identificam visualmente o conteúdo gerado por IA e lembram que a decisão é clínica.

12. Testes

Projecto	Comando	Âmbito
Backend (Jest)	`npm test`	Serviços de autenticação, consultas, prescrições, exames, agenda, justificativos e IA. Cerca de 40 testes cobrem as regras clínicas e a IA, com respostas Gemini simuladas.
Backend (e2e)	`npm run test:e2e`	Arranque da aplicação e rotas principais.
Frontend (Vitest)	`npm test`	AuthContext, cliente da API, exportações e renderização de Markdown.

O que os testes de IA garantem: o contexto enviado não contém dados identificativos; a IA não baixa a prioridade abaixo do piso; as alergias são detectadas com a IA em baixo; os valores de pânico mantêm o exame crítico; a cadeia de reserva funciona em caso de sobrecarga e timeout; um médico não acede às conversas de outro; o enfermeiro não vê a análise de exames.

13. Segurança

Ameaça	Mitigação no LeeVida
Acesso indevido	JWT obrigatório; RBAC no servidor em todas as rotas; regras finas nos serviços (assinatura pelo emissor, dados `/me` identificados pelo token).
Roubo de sessão	Access token de 15 min só em memória; refresh token em cookie `httpOnly`, rodado e guardado como hash.
Senhas fracas ou expostas	bcrypt; política de 8+ caracteres com letras e números; senhas provisórias de troca obrigatória.
Força bruta e abuso	Rate limiting global (100/min) e próprio da IA (20/min).
Injecção e dados inválidos	ValidationPipe com whitelist; Prisma com consultas parametrizadas.
Ataques no navegador	Helmet (cabeçalhos HTTP), CORS restrito à origem do frontend.
Fuga de dados clínicos	Ocultação de resultados ao enfermeiro; IA com dados anonimizados; auditoria de consultas ao prontuário e exportações.
Perda de histórico	Sem eliminação de pacientes nem de contas (arquivo e desactivação); `onDelete: Restrict`.

14. Implantação em produção

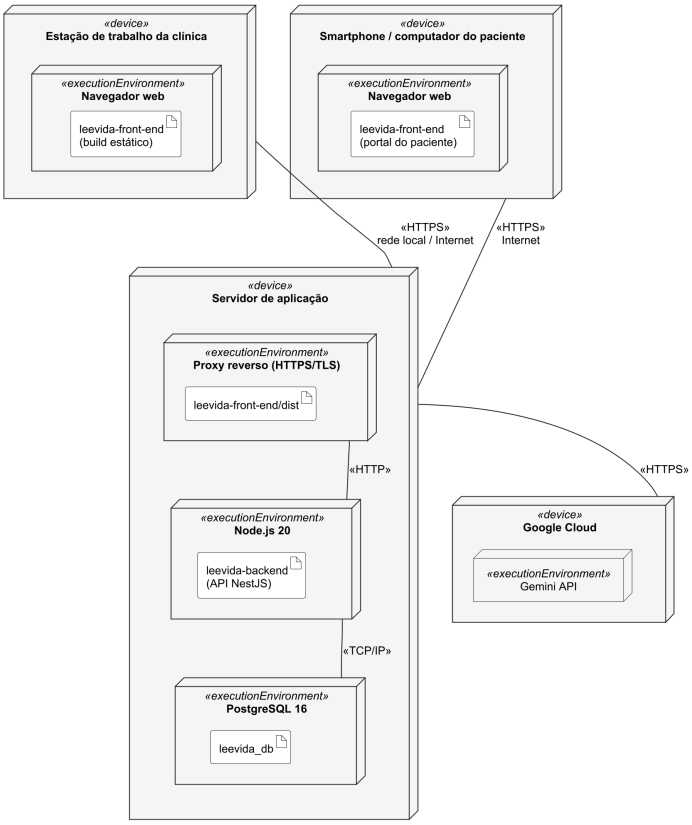


Diagrama de implantação recomendado.

1. **Servidor**: Linux (ou Windows Server) com 2 vCPU, 4 GB de RAM e 40 GB SSD, local na clínica ou alugado na nuvem. Instalar Node.js 20+, PostgreSQL 16+ e um proxy reverso (Nginx ou Caddy).
2. **Base de dados**: criar utilizador e base próprios, sem acesso pela rede externa.
3. **API**: `npm ci`, `npx prisma migrate deploy`, `npm run build` e executar `node dist/main` com um gestor de processos (systemd ou PM2). Definir `NODE_ENV=production`, segredos JWT novos, `CORS_ORIGIN` com o domínio real e `TRUST_PROXY=1`.
4. **Frontend**: `VITE_API_URL=https://dominio/api/v1 npm run build` e servir a pasta `dist/` pelo proxy, com fallback para `index.html`.
5. **HTTPS**: certificado TLS (por exemplo, Let's Encrypt) no proxy. Desactivar a compressão para o caminho SSE `/api/v1/notifications/stream`, ou garantir que o proxy não faz buffering.
6. **Não correr o seed** em produção. Criar o primeiro administrador manualmente e desactivar as contas de demonstração.
7. **Cópias de segurança**: `pg_dump` diário, cifrado e guardado fora do servidor, com teste periódico de reposição.
8. **IA**: activar a facturação da API Gemini e configurar `GEMINI_API_KEY`.

Itens pendentes conhecidos

Paginação nas listagens, fotos de perfil em armazenamento de ficheiros (hoje em base64 na base de dados), Redis para notificações com várias instâncias, sessão única por utilizador, monitorização de erros, testes ponta a ponta no navegador e recuperação de senha por e-mail (exige SMTP). Para produção, convém também ocultar o Swagger.

15. Guias práticos de evolução

15.1 Acrescentar um campo à base de dados

1. Editar o modelo em `prisma/schema.prisma`.
2. `npx prisma migrate dev --name descricao_curta`: gera a migração e actualiza o cliente.
3. Acrescentar o campo ao DTO (com validações `class-validator`) e ao serviço.
4. Acrescentar o campo ao tipo em `leevida-front-end/src/types` e ao formulário ou listagem.
5. Se o campo for exportável, acrescentá-lo a `utils/exportColumns.ts`.

15.2 Acrescentar um endpoint

```
// exams.controller.ts
@Roles(Role.MEDICO, Role.ADMIN)
@Patch(':id/cancel')
cancel(@Param('id') id: string, @CurrentUser() user: RequestUser) {
  return this.examsService.cancel(id, user);
}

// exams.service.ts
async cancel(id: string, actor: RequestUser) {
  const exam = await this.findById(id);              // lança 404 se não existir
  if (exam.status === 'CONCLUIDO') throw new ForbiddenException('Exame já concluído');
  const updated = await this.prisma.exam.update({ where: { id }, data: { status: 'CANCELADO' }
});
  await this.auditLogsService.log(actor, 'CANCELOU_EXAME', 'exam', id);
  return updated;
}
```

O exemplo é ilustrativo: o estado `CANCELADO` exigiria acrescentá-lo primeiro à enumeração `ExamStatus` (guia 15.1). Depois, acrescentar a função em `services/api/exams.ts` no frontend e um teste no `*.service.spec.ts`.

15.3 Acrescentar uma página

1. Criar `src/pages/NovaPage.tsx`.
2. Registar a rota em `App.tsx` dentro de `DashboardLayout`, com `<ProtectedRoute roles={[...]}>`.
3. Acrescentar a entrada no menu em `layouts/Sidebar.tsx` para os perfis certos.

15.4 Ajustar regras clínicas

Alterar os limiares em `src/ai/clinical-rules.ts` e actualizar os testes em `ai.service.spec.ts`. Qualquer alteração deve ser validada pela direcção clínica.

16. Resolução de problemas

Sintoma	Causa provável e solução
`EADDRINUSE :3000` ao arrancar a API	Outra aplicação usa a porta. Usar `PORT=3001` e ajustar `VITE_API_URL` e `CORS_ORIGIN`.
Erros de CORS no navegador	O Vite mudou para outra porta (5174) porque a 5173 estava ocupada. Arrancar com `npx vite --strictPort` ou alinhar `CORS_ORIGIN`.
Sessão perdida ao recarregar a página	O cookie de refresh não é enviado: confirmar que o frontend e a API usam o mesmo esquema (http/https) e que `CORS_ORIGIN` está correcto.
`P1001` do Prisma	PostgreSQL inacessível: verificar se o serviço ou contentor está a correr e a porta em `DATABASE_URL`.
IA responde "indisponível"	Sem `GEMINI_API_KEY`, limite de pedidos atingido, ou todos os modelos em sobrecarga (503). O sistema continua com as regras locais; tentar de novo mais tarde.
Notificações não chegam em tempo real	Proxy a comprimir ou a fazer buffering do SSE. Desactivar para `/notifications/stream`.
Processo do servidor de desenvolvimento não termina (Windows)	Terminar pelo PID: `netstat -ano | findstr :3000` e `taskkill /PID <pid> /F`.